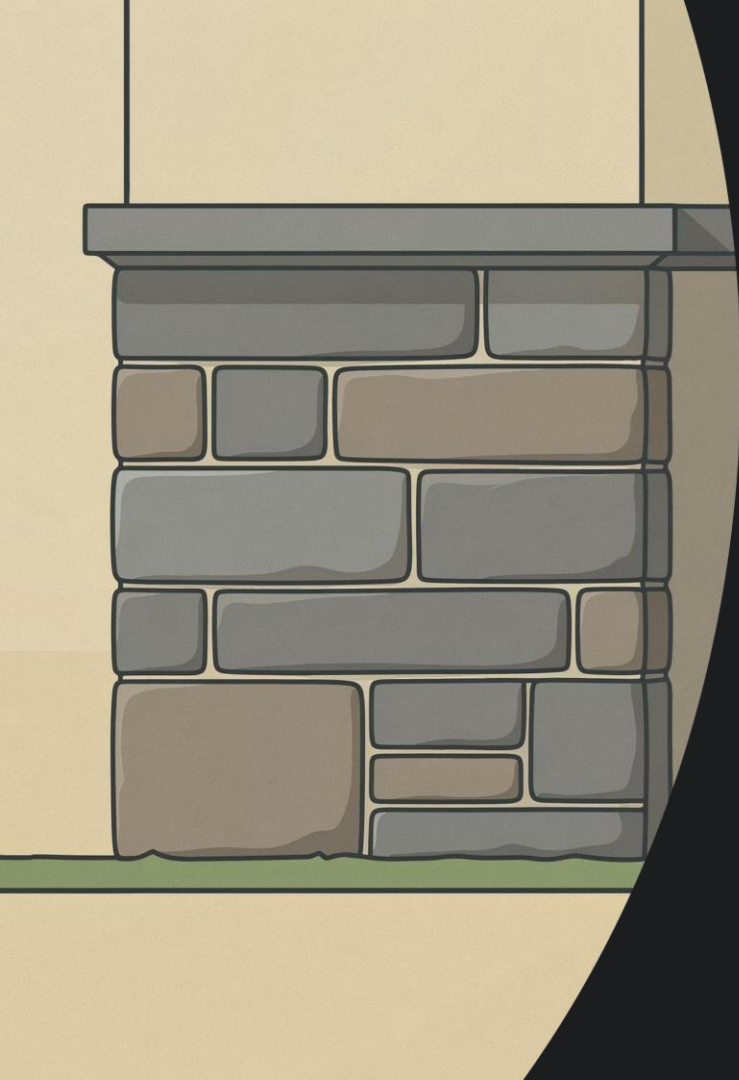




Preparando o Ambiente: Desenvolvendo com C# e .NET

Um guia completo para configurar seu ambiente de desenvolvimento, entender a plataforma .NET e criar seu primeiro programa em C#.



CAPÍTULO 1

O Alicerce Tecnológico

Entenda a relação entre C# e .NET, seus componentes essenciais e como a plataforma funciona por dentro.

O que é a plataforma .NET?

Plataforma Completa

Desenvolvida pela Microsoft, integra runtime, bibliotecas, ferramentas e padrões em um único ecossistema coeso.

Versátil

Permite construir desde aplicações de console simples até sistemas corporativos em nuvem, APIs e serviços web.

Multiplataforma

Roda em Windows, Linux e macOS, com suporte a contêineres e ambientes de nuvem como Azure e AWS.



C# e .NET: Qual a relação?

C# é a linguagem de programação moderna, expressiva e orientada a objetos criada pela Microsoft. **.NET** é o ecossistema que compila, interpreta e executa esse código.



💡 Pense assim: C# é o **idioma** que você fala, e .NET é o **país** com toda a infraestrutura necessária para que essa comunicação aconteça.

A evolução do .NET

.NET Framework

Surgiu no início dos anos 2000 exclusivo para Windows, com foco em aplicações desktop e web legadas.

.NET Moderno

A partir do .NET 5, unificou as plataformas em uma única versão rápida, moderna e com lançamentos anuais.

1

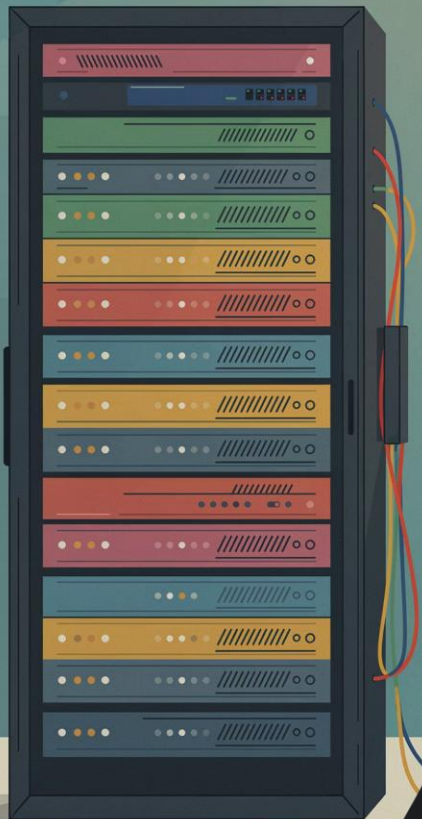
2

3

.NET Core

Reescrita multiplataforma e open source, trazendo performance e flexibilidade para Linux e macOS.

O compilador Roslyn transforma o código C# em bytecode IL, que o runtime (CLR) executa com altíssima eficiência.



Componentes Essenciais para Web



ASP.NET Core

Framework para construir APIs RESTful, aplicações MVC e Razor Pages com alta performance.



Kestrel

Servidor web embutido no .NET, com suporte nativo a protocolos HTTP/1.1, HTTP/2 e HTTP/3.



.NET SDK

O orquestrador central que inclui o compilador, a CLI, templates e todas as ferramentas de build.

SDK vs. Runtime

SDK — Para Desenvolvedores

Inclui compilador, CLI, templates de projeto e ferramentas de build.

Tudo que você precisa para criar software.

Runtime — Para Produção

Apenas o motor de execução do código compilado. Usado em servidores para **rodar** a aplicação já pronta.

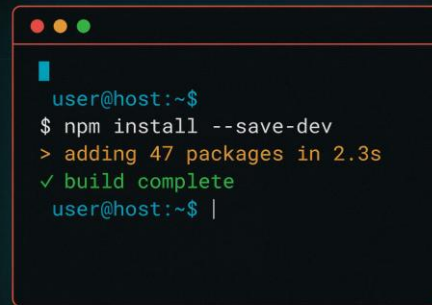
Qual devo instalar?

Para desenvolvimento, a escolha sempre será o **SDK** — ele inclui o Runtime automaticamente. O Runtime standalone é utilizado apenas em ambientes de produção onde não há necessidade de compilar código.

A importância da CLI

A **Command Line Interface** do .NET oferece poder total no terminal, independente de qualquer IDE ou interface gráfica.

- `dotnet new` — cria um novo projeto a partir de um template
- `dotnet build` — compila o projeto
- `dotnet run` — executa a aplicação
- `dotnet publish` — prepara para produção

A terminal window with a dark background and a red border. It shows the execution of the 'npm install --save-dev' command. The output indicates that 47 packages were added in 2.3 seconds and that the build is complete. The prompt 'user@host:~\$' is visible at the top and bottom of the terminal.

```
user@host:~$  
$ npm install --save-dev  
> adding 47 packages in 2.3s  
✓ build complete  
user@host:~$ |
```


CAPÍTULO 2

Escolhendo suas Ferramentas

Conheça o universo das IDEs disponíveis para desenvolvimento C# e escolha a mais adequada ao seu fluxo de trabalho.



O universo das IDEs



Visual Studio

IDE robusta e completa da Microsoft. Ambiente integrado para desenvolvimento, depuração e publicação.



VS Code

Editor leve e extensível. Com o C# Dev Kit, torna-se uma opção poderosa para projetos modernos.



Rider (JetBrains)

IDE paga e multiplataforma com excelente suporte a C#, muito usada por equipes profissionais.

Visual Studio vs. VS Code



Visual Studio:
IDE completa.
Projetos
corporativos.

Janela complexa,
Windows/Mac.
Depurador avançado.
Mais pesado, mais
workloads.



VS Code:
Editor leve.
Projetos
modernos, web.

Multiplataforma.
Extensões C# Dev Kit.
Menor memória,
rápido.

Como escolher?

Se você está começando e quer uma experiência guiada com máximo suporte, opte pelo **Visual Studio**. Para projetos web modernos com maior controle do ambiente, o **VS Code** é excelente.

Neste curso, usaremos o **Visual Studio Community 2022** — gratuito e completo.

Por que o Visual Studio 2022?



Debugger de Alto Nível

Breakpoints, inspeção de variáveis e execução passo a passo integrados nativamente.



Gerenciamento de Workloads

Instalação simplificada de componentes por conjunto de funcionalidades, sem configuração manual.



Community Gratuito

Versão completa e gratuita para estudantes, projetos open source e pequenas equipes.

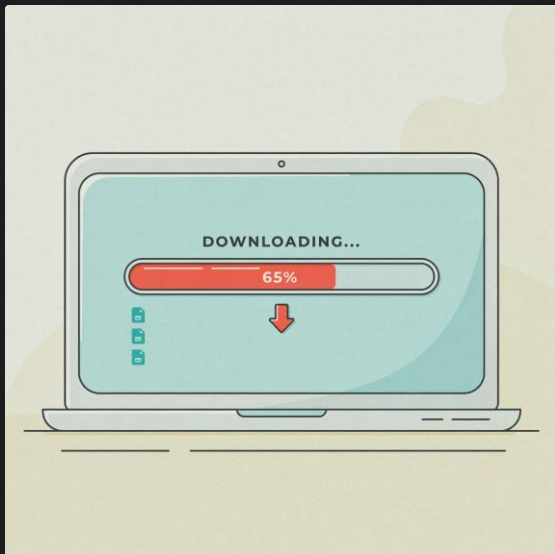




CAPÍTULO 3

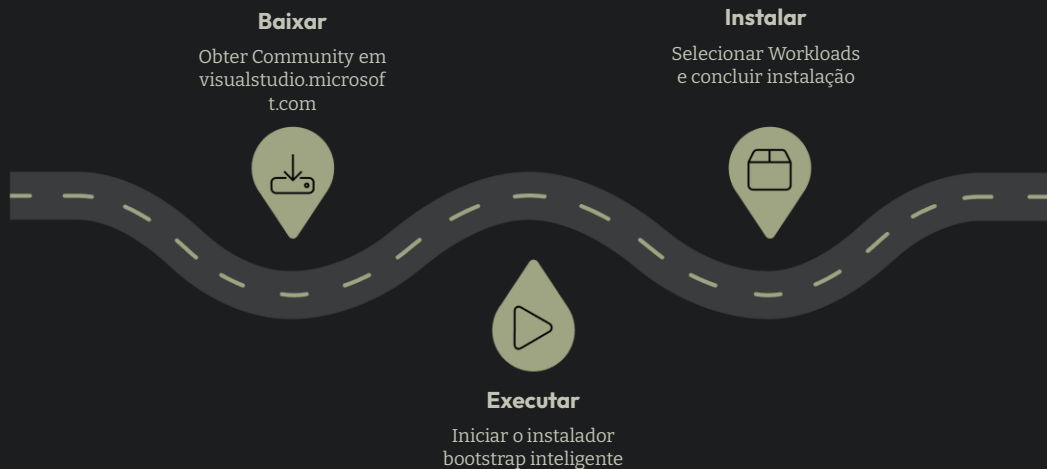
Mão na Massa: Instalação

Passo a passo para instalar o Visual Studio Community 2022 com todos os componentes necessários para o desenvolvimento web com C#.



O Instalador do Visual Studio

O instalador funciona como um **bootstrap inteligente** que baixa e gerencia todos os componentes do Visual Studio.



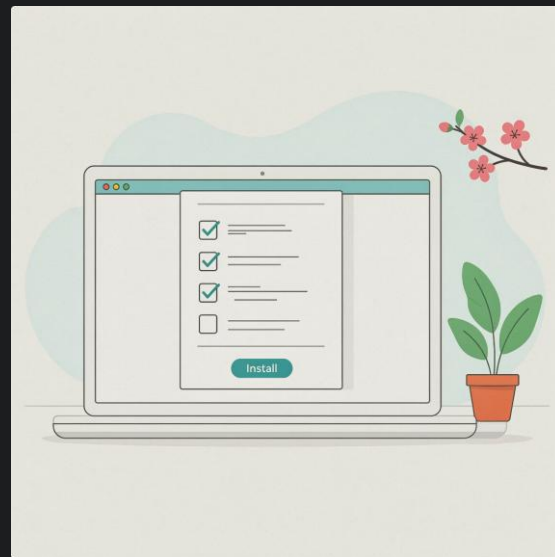
Seleção de Workloads: Web

O que é um Workload?

Um **workload** é um conjunto agrupado de ferramentas, SDKs e templates voltados a um tipo específico de desenvolvimento. Ao invés de instalar componente por componente, você seleciona o pacote adequado.

Para desenvolvimento web com C#, marque:

- ☒ **Desenvolvimento ASP.NET e Web** — inclui automaticamente o .NET SDK, templates de projeto web, Kestrel e ferramentas de publicação.



Seleção de Workloads: Desktop

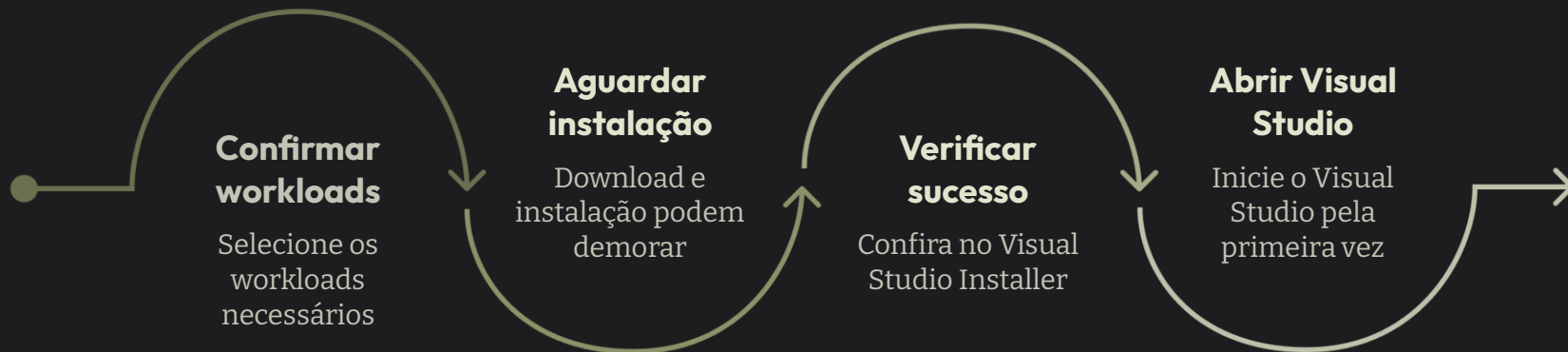


Desenvolvimento para Desktop com .NET

Embora o foco seja web, instalar este workload adiciona suporte a **aplicações console** e **WPF**, úteis para nossos primeiros programas e exercícios práticos.

- Habilita templates de Console App
- Suporte a Windows Forms e WPF
- Otimize o espaço: desmarque componentes que não usará

O processo de instalação



Certifique-se de ter uma boa conexão com a internet — o download pode ultrapassar 5 GB dependendo dos workloads selecionados. Evite erros comuns como desmarcar o SDK acidentalmente.



CAPÍTULO 4

Validando o Ambiente

Antes de escrever qualquer linha de código, confirme que o SDK foi instalado corretamente e que o ambiente está pronto.

O comando dotnet

Abra o **Prompt de Comando** ou o **PowerShell** e execute os comandos abaixo para verificar a instalação:

```
dotnet --version  
dotnet --list-sdks  
dotnet --list-runtimes
```

Se os SDKs instalados aparecerem na listagem, o ambiente está configurado corretamente e pronto para uso.

A screenshot of a terminal window with a dark blue background and white text. The window has a title bar with three colored buttons (red, yellow, green) on the left. The text inside the terminal shows a user at a macbook prompt running 'npm run build'. The output shows three steps: 'Compiling assets...', 'Bundling modules...', and 'Optimizing styles...'. Below these, it says 'Build complete' with a green checkmark, followed by '12 files processed' and '0.8s elapsed'. The prompt returns to 'user@macbook:~\$'.

```
user@macbook:~$  
$ npm run build  
  
> Compiling assets...  
> Bundling modules...  
> Optimizing styles...  
  
✓ Build complete  
> 12 files processed  
> 0.8s elapsed  
user@macbook:~$
```

Solucionando Problemas

Erro: comando não reconhecido

O .NET SDK não foi instalado ou as **variáveis de ambiente** não foram configuradas. Reinicie o terminal ou o computador após a instalação.

Corrigindo via Instalador

Abra o **Visual Studio Installer**, clique em *Modificar* e garanta que o workload correto está selecionado com o .NET SDK incluso.

Instalação Manual do SDK

Se necessário, baixe o .NET SDK diretamente em **dot.net** e instale separadamente, garantindo a inclusão no PATH do sistema.

CAPÍTULO 5

Criando seu Primeiro Projeto

Com o ambiente validado, chegou a hora de criar sua primeira aplicação C# e imprimir o clássico "Hello, World!" na tela.

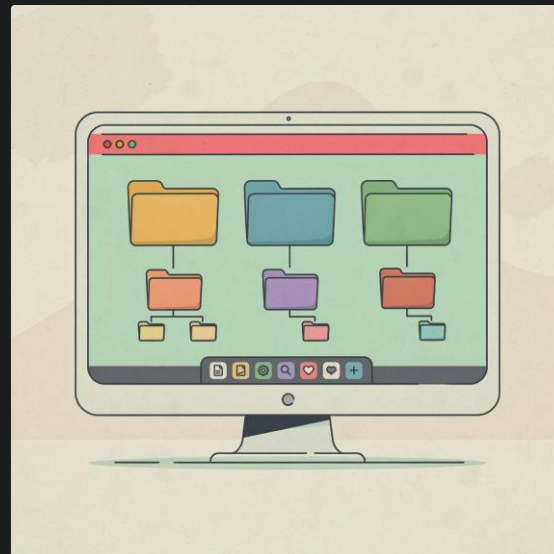


Criando a Solution

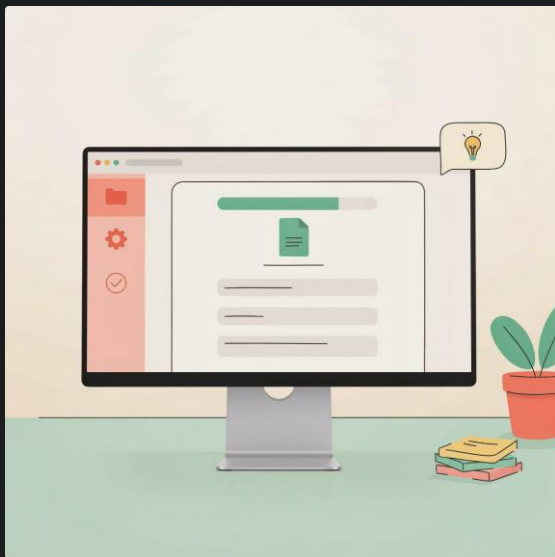
No Visual Studio, uma **Solution** é o contêiner de mais alto nível que organiza um ou vários projetos relacionados em uma única estrutura gerenciável.

- Permite agrupar projetos interdependentes
- Facilita o build e debug de múltiplos projetos
- Organiza a estrutura de pastas no disco

❏ Boas práticas: use nomes em PascalCase sem espaços para Solutions e Projetos.



Criando um Console App



Passo a passo no Visual Studio

1. Clique em **Criar novo projeto**
2. Busque por **Console App**
3. Selecione a versão **sem** ".NET Framework" no nome
4. Defina o nome do projeto (ex: HelloWorld)
5. Confirme a versão do .NET mais recente



⚠️ Atenção: evite o template **.NET Framework** — ele é legado e limitado ao Windows.

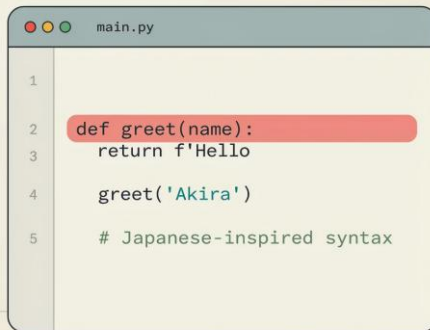
O arquivo Program.cs

O ponto de entrada da aplicação

O `Program.cs` é onde a execução começa. Com o .NET 6+, a estrutura foi simplificada com **Top-Level Statements**:

```
// Hello, World! em C# moderno  
Console.WriteLine("Hello, World!");
```

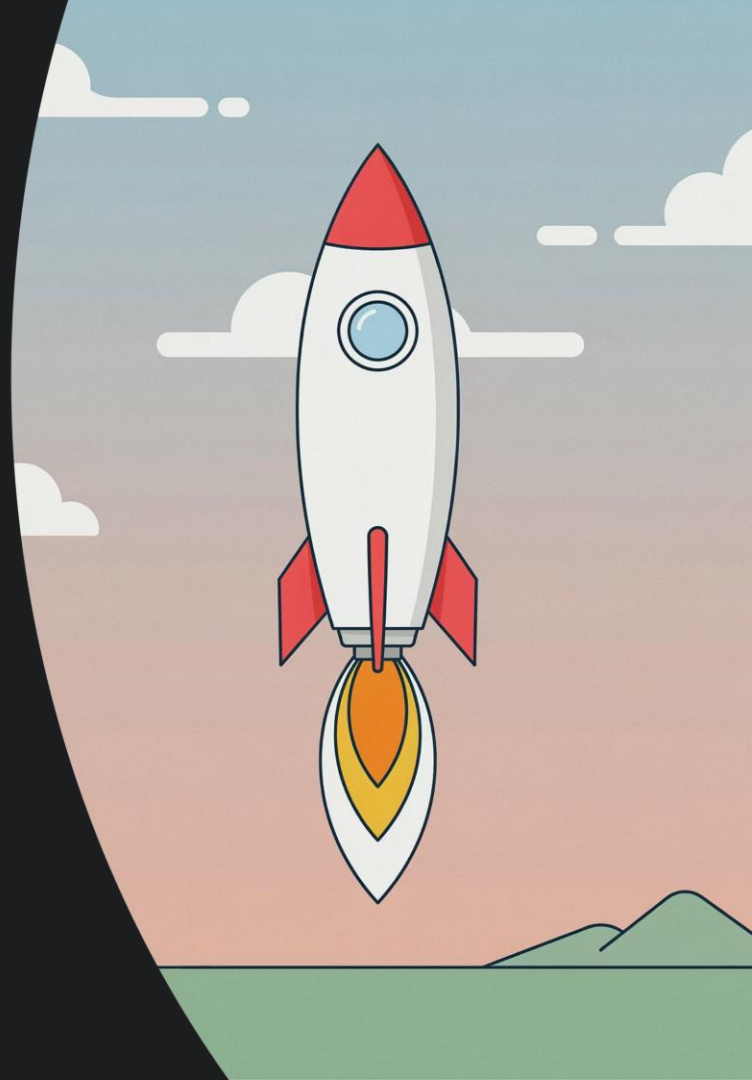
O compilador entende que este é o ponto de entrada e executa o código sequencialmente, linha por linha, do topo ao fim do arquivo.



CAPÍTULO 6

Explorando o Ecossistema NuGet

O NuGet é o gerenciador de pacotes oficial do .NET — sua porta de entrada para milhares de bibliotecas externas.



O papel do NuGet no .NET

1 Busca e instalação de pacotes

Acesse **nuget.org** ou use o Gerenciador de Pacotes do Visual Studio para encontrar bibliotecas e adicioná-las com um clique.

2 Gerenciamento de dependências

O NuGet resolve automaticamente versões e dependências transitivas, registrando tudo no arquivo `.csproj` do projeto.

3 Sem reinventar a roda

Use pacotes consolidados pela comunidade para autenticação, acesso a banco de dados, serialização JSON e muito mais.

Recapitulando a Jornada

01

Entendemos o Ecossistema

Relação entre C# e .NET, SDK vs Runtime, CLI e ASP.NET Core.

02

Escolhemos as Ferramentas

Comparamos IDEs e justificamos o Visual Studio Community 2022.

03

Instalamos e Validamos

Workloads selecionados, ambiente verificado com `dotnet --version`.

04

Criamos nosso Primeiro Programa

Console App com "Hello, World!" e exploração do Program.cs.

An illustration on the left side of the slide shows a perspective view of a dark asphalt road with a dashed white center line, receding into the distance. The road is flanked by green rolling hills. In the background, a large orange sun is setting or rising, creating a warm orange and red sky with soft, horizontal clouds. The entire scene is framed within a large, dark grey circular shape that occupies the right half of the slide.

Próximos Passos



Aprofunde-se em C#

Variáveis, tipos, condicionais, loops e orientação a objetos são os próximos temas fundamentais.



Explore o ASP.NET Core

Crie sua primeira API Web com Controllers, rotas e retorno de dados JSON.



Pratique com NuGet

Adicione pacotes ao seu projeto e experimente bibliotecas da comunidade .NET.